

Tarea Semana 3: Simulador y Taquilla del Teatro Nacional

Funciones, Estructuras de Control y Manejo Robusto de Excepciones

Curso de Programación en Python

Tiempo de entrega: 1 semana

1. Introducción

¡Bienvenido a la evaluación de la Semana 3! A lo largo de estas semanas has aprendido a declarar variables, transformar textos, tomar decisiones con condicionales complejos y controlar flujos repetitivos mediante bucles.

En esta etapa daremos un salto hacia la programación profesional mediante dos pilares:

1. **Funciones:** Para modularizar, estructurar y reutilizar tu código de forma limpia.
2. **Manejo de Excepciones (try-except-else-finally):** Para construir software robusto que no se detenga de forma inesperada ante entradas erróneas del usuario o cálculos matemáticos imprevistos.

En el mundo real, los usuarios cometen errores constantemente. Tu objetivo en esta tarea es programar un sistema interactivo que guíe al usuario con paciencia, valide de forma implacable cada dato y garantice estabilidad absoluta.

2. Objetivos de Aprendizaje

Al finalizar esta tarea, serás capaz de:

- Diseñar funciones con parámetros y retornos claros, gestionando correctamente el alcance (scope) local y global de las variables.
- Anticipar y capturar excepciones específicas como `ValueError`, `TypeError` y `ZeroDivisionError`.
- Implementar flujos avanzados de control de errores utilizando las cláusulas `else` y `finally`.
- Forzar la validación de datos lógicos de negocio utilizando la palabra clave `raise`.
- Crear menús interactivos indestructibles que controlen el flujo del programa mediante bucles de reintento.

3. Enunciado del Proyecto: El Simulador Teatral

Vas a desarrollar un programa interactivo de consola llamado **Simulador de Eventos del Teatro Nacional**. El sistema consta de dos grandes módulos: la venta/taquilla de una reserva individual y la simulación financiera basada en proyecciones físicas del teatro.

4. Requisitos Funcionales y Estructura del Código

Tu programa debe estar completamente modularizado en funciones. A continuación se detallan los componentes obligatorios que debes programar:

4.1. Parte 1: Funciones de Validación de Datos (Manejo de Excepciones)

Debes implementar funciones específicas para solicitar y retornar cada dato del usuario de forma segura. Cada función debe contener un bucle `while True` y bloques `try-except` para repetir la solicitud hasta que el dato introducido sea impecable.

4.1.1. 1. Validación del Nombre del Comprador

- Debe solicitar el nombre completo.
- **Validación con `raise`:** Si el string está vacío (tras usar `.strip()`), debe lanzar un `ValueError` indicando que el nombre no puede quedar vacío.
- El programa debe retornar el nombre formateado usando `.title()`.

4.1.2. 2. Validación de la Zona del Teatro

El teatro tiene tres zonas con precios fijos por asiento. En lugar de usar listas, debes validar la selección mediante un menú numérico estricto:

- **Zona 1 (Platea):** €25,000
- **Zona 2 (Palco):** €15,000
- **Zona 3 (Galería):** €8,000

La función debe pedir el número de zona (1, 2 o 3). Si el usuario introduce letras, `int()` lanzará un `ValueError` que debes atrapar. Si el número no es válido, debes lanzar un `ValueError` personalizado con `raise`. La función debe retornar el precio base correspondiente a la zona elegida.

4.1.3. 3. Validación de Cantidad de Asientos y Edad

- **Edad:** Debe pedir la edad del comprador (entero). Validar con `raise` que esté entre 15 y 100 años. Si es menor de 65 años, el descuento de Ciudadano de Oro es de 0%; si tiene 65 o más, el descuento es del 30%.
- **Asientos:** Debe pedir cuántas entradas desea comprar (entero). Validar que la cantidad sea mayor a 0 y menor o igual a 10 (límite por persona).

4.2. Parte 2: Cálculos Financieros y Cláusulas Avanzadas

4.2.1. 1. Cálculo del Costo Total (Uso Obligatorio de `else` y `finally`)

Debes crear una función llamada `calcular_total(precio_base, cantidad, descuento_adulto_mayor)` que realice la operación matemática.

- Dentro del bloque `try`, realiza el cálculo: $Subtotal = precio_base \times cantidad$. Luego aplica el descuento correspondiente.

- Si ocurre algún error inesperado de tipos (`TypeError`), captúralo.
- Usa el bloque `else` para imprimir un mensaje de confirmación que indique: `¡Cálculo procesado con éxito!`.
- Usa el bloque `finally` para imprimir siempre el mensaje: `¡Cierre de transacción de taquilla!`.
- La función debe retornar el monto final calculado.

4.2.2. 2. Simulación de Distribución de Costos (Manejo de `ZeroDivisionError`)

Para planificar eventos, los productores necesitan calcular cuánto dinero del costo de producción le corresponde financiar a cada asiento vendido. Debes implementar la función: `calcular_costo_distribucion(asientos_vendidos)`.

- Debe calcular: $Costo\ por\ asiento = costo\ producción / asientos\ vendidos$.
- Si el usuario ejecuta la simulación con 0 asientos vendidos, el programa lanzará un `ZeroDivisionError`. **Debes capturar este error específicamente** para evitar que el programa colapse, mostrando un mensaje que explique que no se puede prorratear un costo entre cero asistentes, y retornar 0,0.

—

4.3. Parte 3: Menú Principal Interactivo (Estructura de Control Global)

El programa principal interactuará con el usuario mediante un menú continuo usando la estructura `match` (o condicionales `if-elif-else` anidados si lo prefieres) dentro de un bucle indestructible.

El menú debe ofrecer las siguientes opciones:

1. **Registrar Nueva Venta Individual:** Invoca secuencialmente a las funciones de validación de nombre, zona, edad, asientos y cálculo de montos. Al finalizar, muestra un desglose detallado en pantalla.
2. **Simulador de Costos de Producción:** Pide al usuario un costo estimado de montaje del show y cuántas entradas proyecta vender, invocando a la función que maneja la división entre cero.
3. **Salir del Sistema:** Da una despedida formal y rompe el bucle para finalizar de forma elegante.

—

5. Ejemplo de Ejecución en Consola

Listing 1: Flujo de ejecución del simulador libre de crasheos

```

1  =====
2      SISTEMA DE GESTION - TEATRO NACIONAL
3  =====
4  1. Registrar Nueva Venta Individual

```

```
5 2. Simular Distribucion de Costos de Produccion
6 3. Salir
7
8 Seleccione una opcion: uno
9 Error: Debe ingresar un numero entero valido.
10
11 Seleccione una opcion: 5
12 Error: Opcion fuera de rango. Seleccione entre 1 y 3.
13
14 Seleccione una opcion: 1
15
16 --- REGISTRO DE VENTA ---
17 Ingrese su nombre completo:
18 Error: El nombre no puede quedar vacio.
19
20 Ingrese su nombre completo: marta solano
21 > Nombre registrado: Marta Solano
22
23 ZONAS DISPONIBLES:
24 1. Platea (c25,000)
25 2. Palco (c15,000)
26 3. Galeria (c8,000)
27 Seleccione el numero de zona: 4
28 Error: Zona invalida. Debe elegir 1, 2 o 3.
29
30 Seleccione el numero de zona: 2
31 > Zona seleccionada exitosamente.
32
33 Ingrese su edad: sesenta
34 Error: Debe ingresar un numero entero valido.
35
36 Ingrese su edad: 68
37 > Descuento aplicable: Ciudadano de Oro (30%)
38
39 Cantidad de asientos a comprar: 12
40 Error: Limite excedido. Maximo 10 asientos por transaccion.
41
42 Cantidad de asientos a comprar: 2
43
44 Calculo procesado con exito.
45 Cierre de transaccion de taquilla.
46
47 --- RESUMEN DE COMPRA ---
48 Cliente: Marta Solano
49 Entradas: 2 asientos en Palco
50 Precio Base Unitario: c15,000.00
51 Descuento Aplicado: 30%
52 Monto Final a Pagar: c21,000.00
53 =====
```

6. Requisitos Técnicos y Criterios de Evaluación

La nota se distribuye en un **40 % el código entregado** y un **60 % la defensa oral individual** del mismo en la sesión interactiva.

6.1. Estructura Obligatoria del Código (40 puntos)

- **Uso correcto de funciones (10 pts):** Todo el código debe estar encapsulado en funciones con parámetros y retornos claros. No debe existir código suelto en el archivo global, a excepción de la invocación al menú principal.
- **Captura específica de Excepciones (10 pts):** Está penalizado con -5 puntos el uso de bloques `except`: huérfanos o genéricos que no definan el tipo de error (`ValueError`, etc.).
- **Uso de Else y Finally (10 pts):** Se evaluará que existan al menos dos flujos donde estas dos cláusulas realicen tareas coherentes (mensajes de éxito y limpieza o cierre de operaciones).
- **Lógica y Restricciones (10 pts):** Cero uso de listas o colecciones. El programa debe funcionar puramente con variables numéricas y strings.

6.2. Defensa del Código (60 puntos)

Durante la sesión de defensa con el profesor o asistente, deberás encender tu pantalla y se te pedirá:

- Provocar intencionalmente un error en tu programa para demostrar cómo responde la estructura `try-except`.
- Responder preguntas teóricas clave, tales como:
 - *”¿Cuál es la diferencia exacta en el flujo de ejecución entre el bloque `else` y el bloque `finally`?”*
 - *”¿Por qué usamos `raise` para validar la edad si Python por sí solo no la considera un error de sistema?”*
 - *”¿Qué es el scope o alcance de una variable dentro de tus funciones de validación?”*

¡Mucho éxito con este nuevo desafío!

Aprender a controlar los errores de manera elegante es el paso definitivo para convertirte en un programador profesional.